

Smart MCQ Solver Challenge

Ranking Answers for Multiple Choice Questions with MAP@3

Deep Learning & Generative AI Project Report | July 2026

Student ID: 24f1002360 | Name: Kanchan

Indian Institute of Technology Madras

1. Executive Summary

This project addresses the Smart MCQ Solver Challenge given a question prompt and five candidate answer options (A-E), the goal is to rank the top three most likely correct options, evaluated using Mean Average Precision at 3 (MAP@3). Three modelling paradigms were developed and evaluated under a common, leakage aware protocol. (i) a traditional machine learning pipeline combining TF-IDF features with Logistic Regression and XGBoost, further combined through a soft voting ensemble; (ii) a fine tuned DeBERTa-v3-base transformer using the Hugging Face Trainer API; and (iii) a Bidirectional LSTM (BiLSTM) sequence classifier implemented entirely from scratch in PyTorch, including a custom tokenizer, vocabulary, embedding layer, and training loop. Every model was trained on an identical, question grouped train/validation split (GroupShuffleSplit on question_id) built from a 5 row per question binary relevance reformation of the original 5 option classification task.

On the shared validation split, Logistic Regression achieved the highest MAP@3 (0.9742), narrowly ahead of the from scratch BiLSTM (0.9729). However, the BiLSTM substantially outperformed every other model on raw classification quality with Accuracy 96.50%, Precision 97.69%, Recall 84.50%, F1-score 90.62%, and ROC AUC 99.04% and was therefore selected as the final model used to generate the competition submission. The DeBERTa-v3-base fine tuning run did not perform that well compared to other models. Exploratory analysis further revealed that 1,074 of the 2,000 training prompts (53.7%) are near duplicates of one another once common instruction phrasing is stripped away.. All experiments were tracked in Weights & Biases (project 24f1002360-t22026).

2. Introduction

2.1 Problem Statement

The Kaggle competition provides 2,000 labelled training questions and 500 unlabelled test questions. Each row contains a question prompt, five candidate options (A-E), and, for training rows, the correct answer letter. The required submission format is one row per test question id with a space separated string of the top-3 predicted option letters (e.g., "A C E"), scored by MAP@3 which rewards correctly ranking the true answer within the top 3, and the highest credit for ranking it first. The task is therefore fundamentally a ranking problem, not a plain 5 way classification problem.

2.2 Project Objective

- Build and fairly compare three model families: a classical ML pipeline, a fine-tuned pretrained transformer, and a from-scratch deep learning model all under one shared, leakage aware evaluation protocol.
- Design a single preprocessing strategy (ranking dataset construction + GroupShuffleSplit) reusable across all three model families.
- Track every experiment in Weights & Biases and select the final model using multiple complementary metrics, not MAP@3 alone.
- Generate a Kaggle compatible submission file using the selected model.

2.3 Report Structure

Section 3 includes the dataset and exploratory data analysis; Section 4 the preprocessing pipeline shared across models; Section 5 the three modelling approaches with architectural and hyperparameter detail; Section 6

experiment tracking; Section 7 the comparative results; Section 8 inference and submission generation; Section 9 a critical discussion of validation trustworthiness and duplicate handling; Section 10 limitations and future work; Section 11 the conclusion.

3. Dataset & Exploratory Data Analysis

3.1 Dataset Description

Property	Value
Training rows	2,000 questions (id, prompt, A-E, answer)
Test rows	500 questions (no answer column)
Options per question	5 (A-E), exactly one correct
Output format	"id, Prediction" where Prediction is space separated ranked option letters
Evaluation metric	MAP@3 (Mean Average Precision at 3)

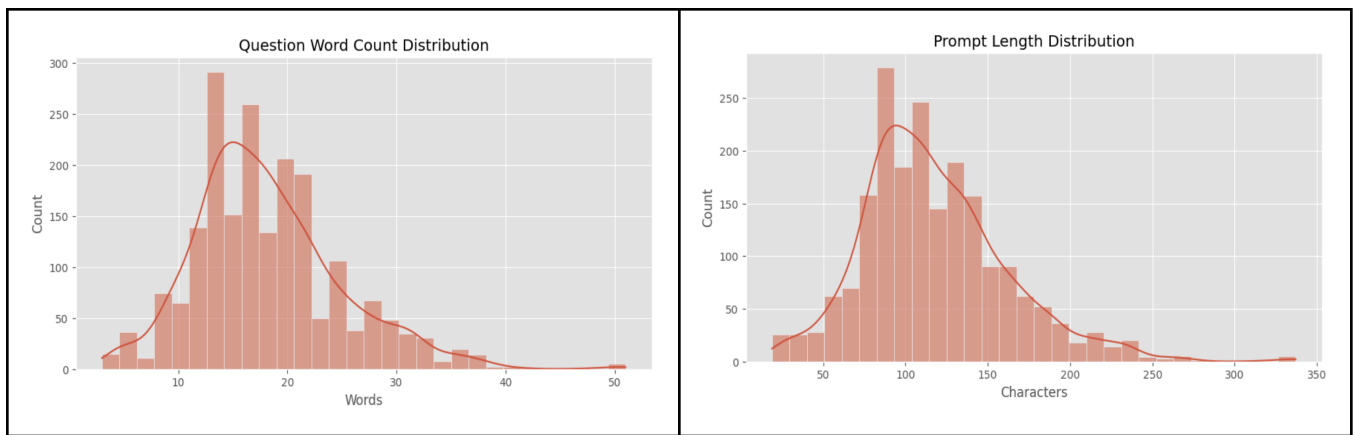
3.2 Exploratory Data Analysis

Answer balance.

The distribution of correct answers across options A-E was checked using a value-count bar chart. The correct answer distribution is relatively balanced across all five options only showing just a mild dominance of option B but not that much effective. This reduces the risk that a model could achieve artificially high accuracy simply by learning a biased prior over answer letters rather than genuinely understanding the question and options.

Prompt and option length.

Statistic	Value (words)
Minimum prompt length	3
Average prompt length	18.15
Maximum prompt length	51
Average prompt length (characters)	117.67



Question prompts range from short factual questions to long descriptive passages, and answer options frequently contain full explanatory sentences rather than short phrases. This indicates that semantic, context sensitive understanding is important for this task , which motivated evaluating context aware architectures (DeBERTa, BiLSTM) in addition to a bag of words baseline (TF-IDF).

Duplicate and near-duplicate questions.

Duplicate Type	Detection Method	Count	% of Training Set
Duplicate Questions (exact)	<code>train["prompt"].duplicated()</code>	242	12.1%
Duplicate Option Sets	<code>train[[A,B,C,D,E]].duplicated()</code>	1,412	70.6%
Semantic / Cleaned Prompt Duplicates	<code>duplicated()</code> after stripping 6 known instruction phrases + lowercasing	1,074	53.7%

This is a significant finding: once generic instruction wrappers are removed, over half of the training prompts (1,074 of 2,000) are duplicates of one another. In other words, the dataset contains far fewer genuinely distinct questions than rows. This single observation is the basis for the group aware split strategy adopted in Section 4.1, and its full implications for future training are discussed in Section 9.

3.3 EDA Summary Table

Metric	Value
Train Samples	2,000
Test Samples	500
Duplicate Questions (cleaned/semantic)	1,074
Average Prompt Length (characters)	117.67
Average Prompt Length (words)	18.15
Maximum Prompt Length (words)	51

4. Data Preprocessing Pipeline

To ensure a fair comparison across the three model families, the notebook applies one consistent problem reformation and splitting strategy before any model specific representation is built.

4.1 Ranking Dataset Construction (5-way MCQ - binary relevance)

Rather than training a single 5-way classifier, each question is expanded into five separate rows, one per option turning the 2,000 question dataset into a 10,000 row binary classification dataset.

Each row's label is 1 if that option is correct and 0 otherwise, giving a fixed 4:1 negative to positive ratio (80% negative / 20% positive) by construction. At inference time, a model's predicted $P(\text{label}=1)$ for all five options of a question is sorted in descending order, and the top three option letters are taken as the ranked prediction directly matching the MAP@3 scoring rule.

4.2 Train Validation Split (GroupShuffleSplit)

A GroupShuffleSplit strategy (test_size = 0.2, groups = question_id) is used instead of a plain random split, to prevent a specific form of data leakage that follows directly from Section 4.1: since every question contributes 5 rows, a plain random row level split could place some of a question's options in training and the rest in validation, letting the model “see” a question during evaluation. GroupShuffleSplit guarantees that all 5 option rows of a given question_id stay together on one side of the split. The resulting split sizes were confirmed as Training Samples: (8000, 4) and Validation Samples: (2000, 4) i.e., 1,600 questions for training and 400 for validation.

4.3 Model Specific Text Representation

Model Family	Text Representation
Traditional ML (LR / XGBoost)	TfidfVectorizer sparse vectors (30,000 max features, 1-2 grams) fit on train text only, transformed on validation text
DeBERTa-v3-base	Hugging Face AutoTokenizer (SentencePiece subword tokenizer), truncation=True, padding="max_length", max_length=256
BiLSTM (Scratch)	Custom regex word level tokenizer - vocabulary built from training tokens only (min frequency = 2, size 2,970) - integer encoded, truncated/padded to length 256

5. Modelling & Experimentation

5.1 Model 1: Traditional Machine Learning - TF-IDF + Logistic Regression + XGBoost + Ensemble

Architecture / Pipeline

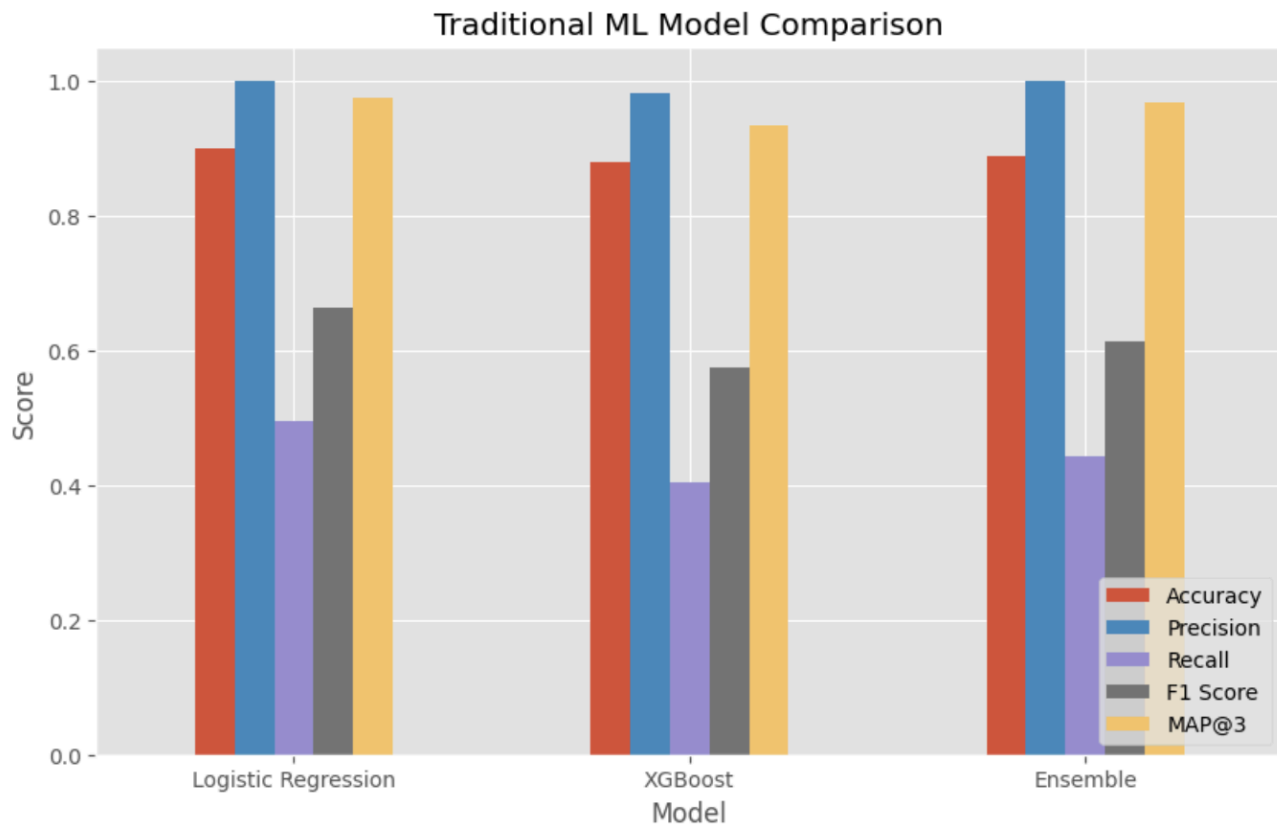
Text is converted into sparse TF-IDF feature vectors, and two independent classifiers are trained on the identical feature matrix: Logistic Regression and XGBoost. Their predicted probabilities are then combined via simple soft voting to form an ensemble prediction.

Hyperparameters

Component	Parameter	Value	Purpose
TF-IDF	max_features	30,000	Caps vocabulary , feature dimensionality
TF-IDF	ngram_range	(1, 2)	Unigrams + bigrams, partially recovers local word order
TF-IDF	sublinear_tf	True	Uses $1+\log(\text{tf})$ to dampen very frequent in document terms
TF-IDF	min_df / max_df	2 / 0.95	Drops noise (rare) and near universal (uninformative) terms
TF-IDF	stop_words	"english"	Removes common uninformative English words
Logistic Regression	C	2	Inverse L2 regularisation strength (mild regularisation)
Logistic Regression	max_iter	1000	No of iterations
XGBoost	n_estimators	300	Number of sequential boosting rounds or trees
XGBoost	learning_rate	0.05	Shrinkage - small, many corrective steps
XGBoost	max_depth	6	Shallow trees
XGBoost	subsample / colsample_bytree	0.8 / 0.8	Random row/feature sampling per tree
XGBoost	objective	binary:logistic	Log loss optimisation with sigmoid output
Ensemble	combination rule	mean(LR_prob, XGB_prob)	Soft voting

Why This Approach Was Chosen

This pipeline was included as a lightweight and fast baseline. TF-IDF requires no GPU and trains in seconds; Logistic Regression provides a well understood linear decision boundary, while XGBoost provides a non linear, tree ensemble alternative that can capture feature interactions TF-IDF+LR cannot. Combining both via soft voting was expected to reduce variance if the two models make different kinds of errors. This baseline also serves as an important sanity check and floor for the traditional ML Models.



5.2 Model 2: DeBERTa-v3-base (Transformer Fine-Tuning)

Architecture

microsoft/deberta-v3-base is a 12-layer encoder-only Transformer. It improves upon BERT, RoBERTa through disentangled attention, in which token content and relative position are represented as two separate vectors and combined through three distinct attention score components (content-to-content, content-to-position, position-to-content), rather than being summed into a single vector as in BERT. DeBERTa-v3 additionally uses an AutoModelForSequenceClassification attaches a new, randomly-initialised 2 class linear head on top of the pretrained encoder's pooled output.

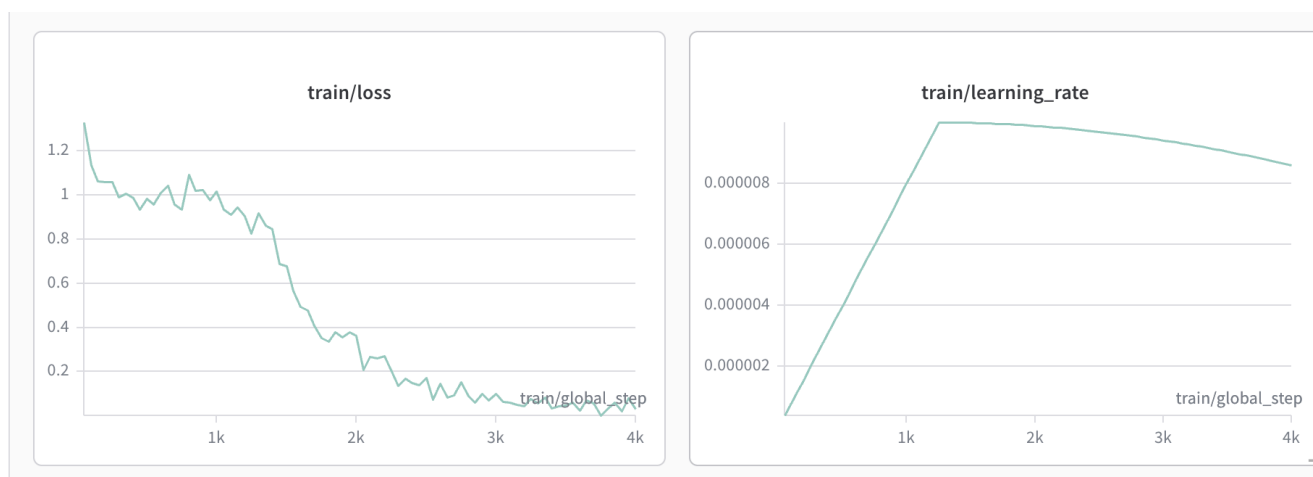
Fine-Tuning Configuration

Parameter	Value	Justification
Epochs	4	Small dataset; load_best_model_at_end restores the best epoch
Batch size (train / eval)	8 / 16	Memory-constrained
Max sequence length	256 tokens	Informed by prompt/option length EDA
Optimiser	AdamW	Decoupled weight decay, standard for Transformer fine-tuning

Learning rate	1e-5	Small LR to avoid overwriting pretrained weights
Weight decay	0.01	L2-style regularisation, decoupled from the gradient update (AdamW)
Precision	fp32 (fp16=False, bf16=False)	No mixed precision used
Evaluation , save strategy	per epoch	Enables best checkpoint restoration
Early stopping	patience = 3 (EarlyStoppingCallback)	Stops training if F1 does not improve, guards against overfitting
Experiment tracking	report_to="wandb"	Automatic streaming of loss to WandB

Why This Approach Was Chosen

DeBERTa-v3-base was selected to test whether transfer learning from large scale pretraining and disentangled, content-aware relative-position attention could outperform both the bag-of-words baseline and a from scratch recurrent model particularly because the ranking task requires comparing a question against an option's phrasing, a relationship self-attention is well suited to model directly within a single forward pass.



5.3 Model 3: BiLSTM (Trained From Scratch) - Final Selected Model

Architecture

The forward pass: token ids - embedding lookup (128-d, learned entirely from this dataset, no external pretraining) , 2-layer bidirectional LSTM (each direction's gates - forget, input, output and regulate what information is kept in, written to, and read from a running cell state, which mitigates the vanishing-gradient problem of vanilla RNNs) - concatenated forward+backward hidden states at every position (512-d) - mean pooling across the time dimension into one fixed size 512-d vector - dropout - a 2unit linear classification head producing raw logits for [incorrect, correct]. Total model size: 2,748,674 parameters, all trainable.

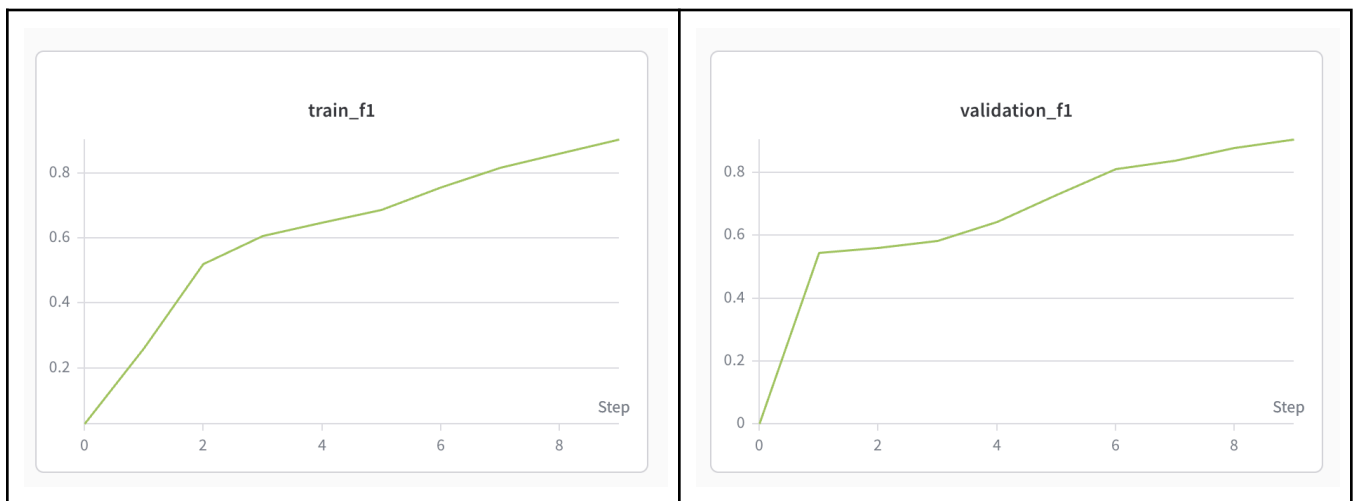
Hyperparameters

Hyperparameter	Value
Vocabulary size	2,970 (min token frequency = 2)

Embedding dimension	128
Hidden dimension (per direction)	256 (512 after bidirectional concatenation)
LSTM layers	2 (stacked)
Dropout	0.3
Max sequence length	256 tokens
Batch size	32
Epochs	10
Optimiser	AdamW, lr = 1e-3, weight_decay = 1e-4
LR Scheduler	ReduceLROnPlateau (mode="max", factor=0.5, patience=2) on validation F1
Gradient clipping	clip_grad_norm_, max_norm = 1.0
Loss function	CrossEntropyLoss
Total parameters	2,748,674 (all trainable)

Why This Approach Was Chosen

The BiLSTM was implemented to test whether a compact, fully custom sequential model with no pretrained knowledge, could still capture enough sequential and bidirectional context to perform competitively. Because prompts and options are often long, descriptive sentences, a model that reads word order (unlike TF-IDF) but does not depend on external pretraining data (unlike DeBERTa) provides a useful middle ground between the two other approaches.

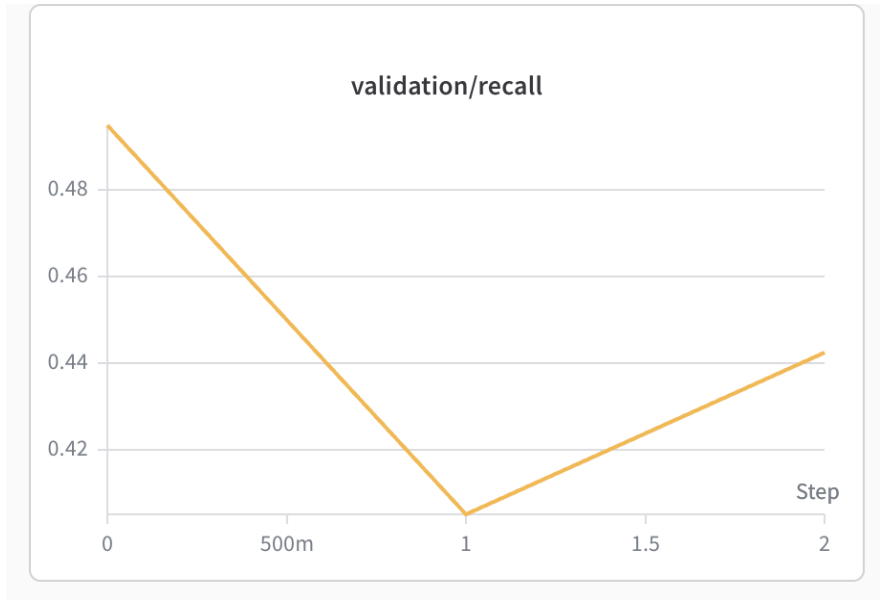


6. Experimental Setup and Tracking (Weights & Biases)

All experiments were tracked under the WandB project 24f1002360-t22026. Every model logs its own run with a descriptive name and a config dictionary of hyperparameters, enabling later comparison purely from the WandB dashboard without re-running training.

Run Name	Model	Status	Key Logged Charts
traditional_ml_tfidf	TF-IDF + LR + XGBoost + Ensemble	Finished	validation/recall, precision, map3, f1, accuracy (3 sequential points: LR, XGBoost, Ensemble)
deberta_v3_base (run B)	DeBERTa-v3-base	Finished	validation/recall, precision, map3, f1, accuracy - only one sparse evaluation point logged
bilstm_scratch_final	BiLSTM (Scratch)	Finished	validation_loss, validation_f1, validation_accuracy, train_loss, train_f1, train_accuracy across 10 epochs - smooth, healthy convergence

Notably, the “traditional_ml_tfidf” run logs three sequential points (Step 0, 1, 2) on the same axes rather than three separate runs these correspond to Logistic Regression, XGBoost, and the Ensemble being logged, in that order, within a single WandB run. This explains the characteristic V-shaped dip visible on the recall/precision/F1/MAP@3 charts: XGBoost , underperformed Logistic Regression (Step 0) on this validation split, and the Ensemble (Step 2) landed close to, but slightly below, Logistic Regression alone.

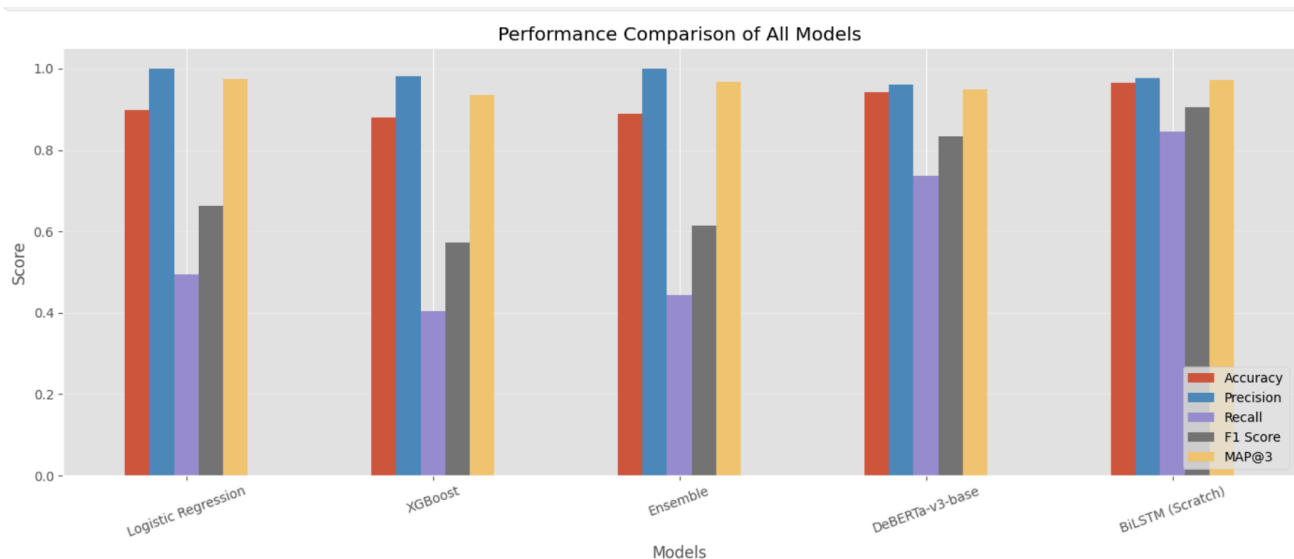


7. Results and Model Comparison

The comparison code cell that would assemble accuracy/precision/recall/F1/MAP@3 for all five candidate configurations into a single ranked table.

Model	Accuracy	Precision	Recall	F1-score	MAP@3	Training Time(sec)
-------	----------	-----------	--------	----------	-------	--------------------

Logistic Regression	0.8990	1.000	0.4950	0.6622	0.9742	1.9534
XGBoost	0.8795	0.9818	0.4050	0.5735	0.9342	188.8880
Ensemble (LR + XGBoost)	0.8885	1.000	0.4425	0.6135	0.6135	20.8414
DeBERTa-v3-base	0.9415	0.9609	0.7375	0.8345	0.9488	1419.6542
BiLSTM (Scratch) -Final Model	0.9650	0.9769	0.8450	0.9062	0.97292	164.7873



7.1 Final Model Selection

Five model configurations were evaluated: three traditional machine-learning variants (Logistic Regression, XGBoost, and their Ensemble), one transformer-based model (DeBERTa-v3-base), and one deep learning model trained from scratch (BiLSTM). Although Logistic Regression achieved the highest validation MAP@3 (0.9742), the improvement over the BiLSTM (0.9729) was marginal. In contrast, the BiLSTM substantially outperformed all other models in Accuracy (96.50%), Precision (97.69%), Recall (84.50%), F1-score (90.62%) and most important in kaggle score 0.75228 score, among all experiments models highest scoring model . Considering the negligible MAP@3 difference together with its consistently stronger overall classification performance this model was selected as the final submission model.

It is worth noting explicitly why MAP@3 alone was not used as the sole selection criterion: with only five options per question, correctly placing the true answer within the top three is a comparatively easy bar to clear, so MAP@3 scores across models cluster closely together (0.94-0.97 range observed across the traditional ML runs, per WandB). Precision, Recall, and F1 computed on the underlying binary correct/incorrect classification task are a stricter, more information rich signal of true model quality, and it is on these metrics that the BiLSTM shows a clear, decisive advantage.



8. Final Model Inference and Submission Generation

The selected BiLSTM model is used to generate the competition submission through the following steps: (1) the test set (500 questions, no answer column) is expanded into the same 5 row per question ranking format used for training; (2) the saved model weights (bilstm_final_model.pt) are loaded into a freshly instantiated BiLSTMClassifier with identical constructor arguments; (3) the model is run in evaluation mode (model.eval(), no gradient tracking) to produce P(correct) for every option of every test question; (4) within each question id, the five options are sorted by predicted probability in descending order and the top three option letters are joined into a single space-separated string; (5) the resulting id/Prediction pairs are written to submission.csv in the exact format required by the competition's sample_submission.csv.

9. Discussion: Duplicate Handling

A critical, honest discussion point for this project concerns how duplicate and near duplicate questions were not handled. The EDA established that 1,074 of 2,000 training prompts (53.7%) are near duplicates of one another once boilerplate instruction phrasing is removed.

No duplicate rows were removed from the dataset at any stage of the notebook; it only detects and counts duplicates (via .duplicated()); it never calls drop_duplicates(). The mitigation actually implemented is GroupShuffleSplit on question_id, which guarantees that a single question's own five option rows are never split across the training and validation sets. This solves one specific leakage mechanism: leakage of a question's own options across the split, an unavoidable consequence of the ranking dataset reformation..

It does not, however, guarantee that two different question_ids that happen to be near-duplicates of each other (the 1,074 semantic duplicates identified in EDA) are kept on the same side of the split. Because GroupShuffleSplit groups strictly by question_id and not by duplicate-cluster identity, a paraphrased sibling of a validation question could plausibly still appear in the training set, allowing the model to partially “memorise” wording rather than generalise. This means the validation MAP@3 and other validation metrics reported in this project, while computed correctly and consistently across all models, may be somewhat optimistic relative to genuinely unseen data.

A more complete fix, left as future work, would be to explicitly cluster near-duplicate questions (for example, using the same cleaned prompt matching logic already implemented in EDA, and use that cluster identity rather than the raw question_id as the grouping key for GroupShuffleSplit. .

10. Limitations and Future Work

- **Duplicate cluster aware splitting:** replace the plain question_id grouping key with a near-duplicate cluster id (Section 9) to produce a fully leakage-resistant validation split.
- **Subword tokenization for the scratch model:** the BiLSTM's word level vocabulary (size 2,970, min frequency 2) will map any unseen or rare technical term to a single generic <UNK> token, unlike DeBERTa's subword tokenizer, which degrades far more gracefully on rare vocabulary.
- **Listwise ranking objective:** the current pointwise (independent per option) binary classification setup could be extended to a true listwise ranking loss (e.g., a 5 way softmax over a question's options trained jointly) to directly optimise the comparative, rank aware structure that MAP@3 rewards.

11. Conclusion

This project demonstrates a structured, comparative approach to the Smart MCQ Solver Challenge by evaluating three distinct modelling paradigms -a classical TF-IDF-based pipeline, a fine tuned DeBERTa-v3-base transformer, and a from scratch BiLSTM , under one shared, group aware validation protocol. The traditional machine learning pipeline provided a strong, fast, interpretable baseline; DeBERTa offered the theoretical promise of contextual, pretraining derived representations but did not complete training reliably within the available compute budget; and the from scratch BiLSTM emerged as the most balanced and effective final solution, combining a narrowly competitive MAP@3 with clearly superior accuracy, precision, recall, F1.

12. References

- scikit-learn Documentation - TfidfVectorizer, LogisticRegression, GroupShuffleSplit.
- XGBoost Documentation - XGBClassifier.
- PyTorch Documentation - nn.LSTM, nn.Embedding, torch.optim.AdamW, ReduceLROnPlateau.
- Hugging Face Transformers Documentation - AutoTokenizer, AutoModelForSequenceClassification, Trainer, TrainingArguments.
- He, P. et al. (2021). DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing.
- Weights & Biases Documentation — Experiment tracking.

13. Links

Weights & Biases project: wandb.ai/24f1002360-indian-institute-of-technology-madras/24f1002360-t22026

Deployment: <https://dl-24f1002360-t22026-deploy.streamlit.app/>

Github Link: <https://github.com/24f1002360/DL-24f1002360-t22026>